

Laboratorio de Estructura de Computadores

ELO 312 - Clase 01

Presentación del curso e Introducción a C

Alejandro Weinstein
alejandro.weinstein@usm.cl

04 de agosto 2026



Profesores

- Martes: Mauricio Solís
- Miércoles: Alejandro Weinstein (coordinador)
- Viernes: Rodrigo Jimenez

Sobre el curso

- Reglamento
- Formar grupos
- Discord
(<https://discord.gg/7sVUtFyubN>)
- Programación de la asignatura
- Guía sesión guiada 1.



Reglamento: sobre los grupos

El trabajo en el laboratorio se realiza en grupos, los cuales serán formados al azar previo a la primera sesión. Solo si todos los estudiantes del paralelo están de acuerdo se podrá de manera excepcional conformar grupos por afinidad considerando las siguientes normas:

- Los grupos deben ser de un máximo de 3 estudiantes.
- No puede haber más de 2 grupos de 2 estudiantes.
- Ningún estudiante puede quedar solo.

Reglamento: sobre las evaluaciones

- Asistencia obligatoria (ver penalizaciones por atraso).
- Sesiones guiadas y evaluadas.
- Tres certámenes escritos e individuales.
- Ver programación en aula.

Reglamento: sobre las evaluaciones

Nota final:

Si la nota promedio de los certámenes es mayor o igual a 60

$$N_F = 0.15E_1 + 0.15E_2 + 0.3E_3 + 0.4C$$

Si la nota promedio de los certámenes es mayor o igual a 50 y menor a 60 puede dar un examen práctico individual (sin internet).

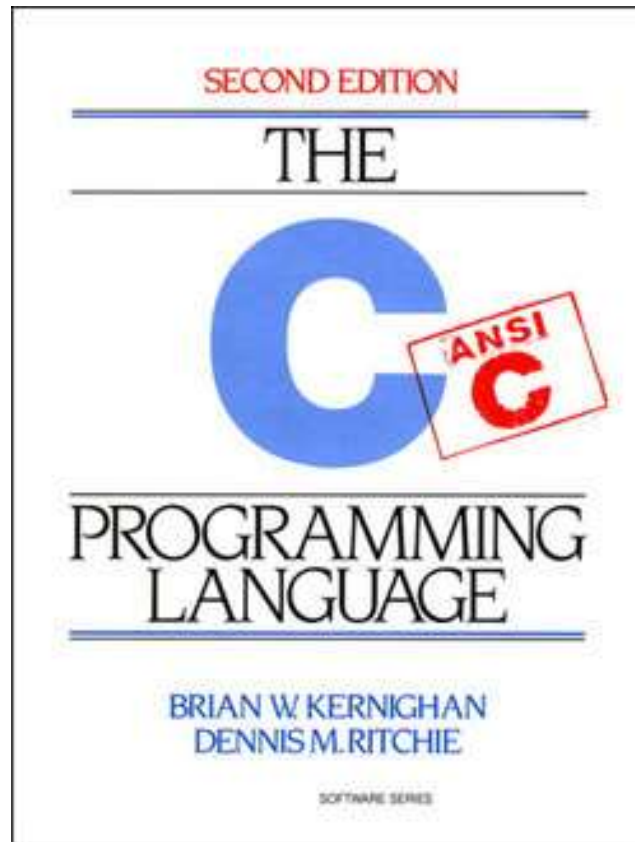
Ver detalles en el reglamento.

“... no puede ser que se espere que el alumno encuentre LA línea de código dentro de un pdf de datasheet o user manual de 1000 páginas.”

Introducción al lenguaje de programación C

- <http://www.cplusplus.com/reference/library/>
- <https://www.cprogramming.com/tutorial/c-tutorial.html>
- <https://www.tutorialspoint.com/cprogramming/index.htm>

K&R



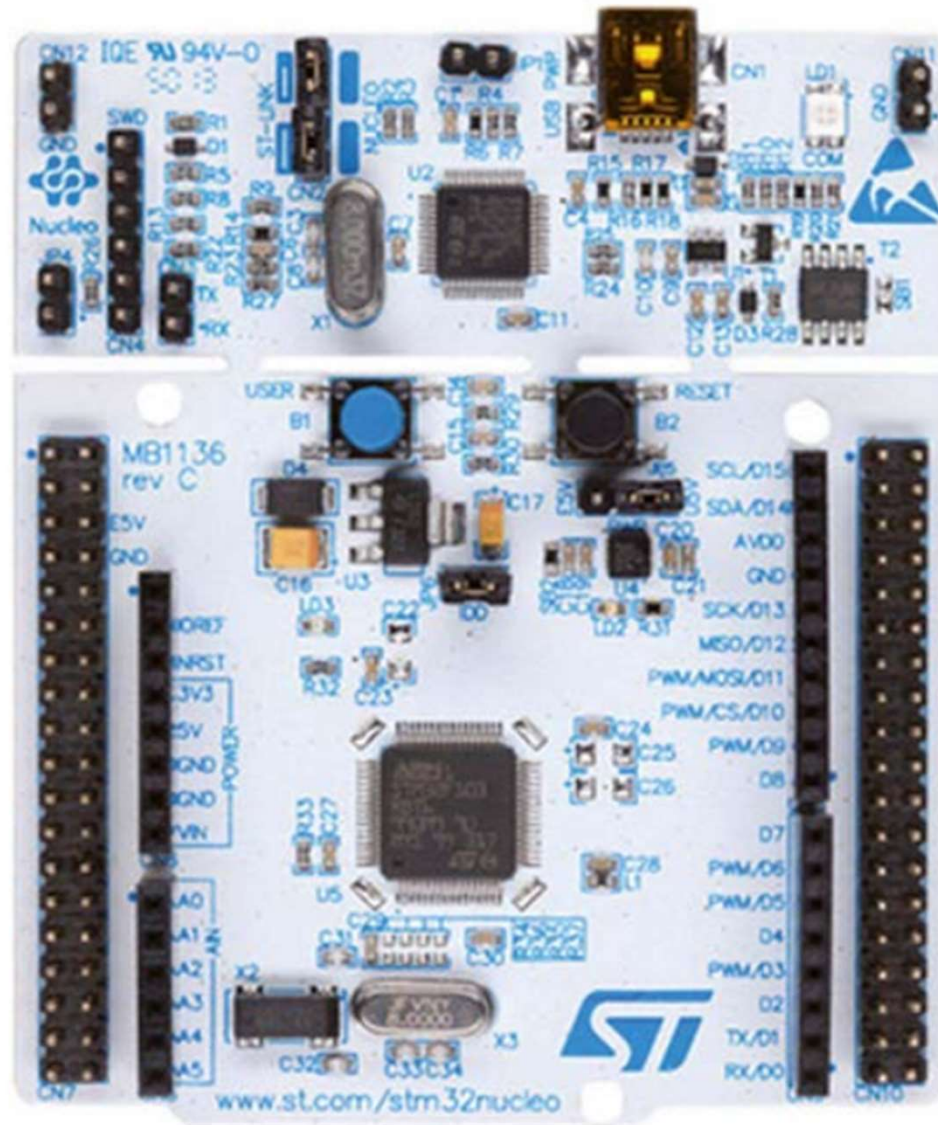
Antecedentes generales

- C es un lenguaje de programación de propósito general para *programación estructurada*
- ¿Por qué C?
 - Control directo del hardware con una sintaxis más legible que ensamblador.
 - Código eficiente en ejecución y uso de memoria.
 - Portabilidad entre distintas arquitecturas de microcontroladores.

Instalar STM32CubeIDE

<https://www.st.com/en/development-tools/stm32cubeide.html>

Nucleo-L476RG



Un programa en C consiste en

- Comandos del preprocesador
- Funciones
- Variables
- Declaraciones y expresiones
- Comentarios

Ejemplo

```
#include <stdio.h>
```

```
int main( ) {  
    /* mi primer  
    programa en C */  
    int edad = 32;  
    printf ("Tengo %d años. \n", edad);  
    getchar( );  
    // comentario en una solo línea  
    return 0;  
}
```

Variables

- Los datos se almacenan en variables.
- Al declarar la variable se debe indicar el tipo de datos. Por ejemplo:
 - char, int y float
- **Algunas** variables también **usan más memoria** para almacenar sus valores.
- Sintaxis: **<tipo> <nombre>;** Por ejemplo:

```
int foo;
```
- Se permite declarar múltiples variables del mismo tipo en la misma línea:

```
char a, b, c, d;
```
- No se puede tener múltiples variables con el mismo nombre, tampoco tener variables y funciones con el mismo nombre.
- Notar la diferencia con, por ejemplo, Python.

El tipo de variable define la cantidad de bytes necesarios para almacenarla

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

`sizeof (type)` retorna el tamaño del tipo de variable, en bytes

inttypes.h

La dependencia del tamaño del tipo de datos en función de la arquitectura puede ser un problema, sobre todo en sistemas embebidos

Fixed width integer	signed	unsigned
8 bit	int8_t	uint8_t
16 bit	int16_t	uint16_t
32 bit	int32_t	uint32_t
64 bit	int64_t	uint64_t

Preprocessor: #define

```
#include <stdio.h>
```

```
#define LENGTH 10
```

```
#define WIDTH 5
```

```
#define NEWLINE '\n'
```

```
int main( ) {
```

```
    int area;
```

```
    area = LENGTH * WIDTH;
```

```
    printf ("value of area : %d", area);
```

```
    printf ("%c", NEWLINE);
```

```
    return 0;
```

```
}
```

Operadores aritméticos

(asumiendo que $A = 10$ y $B = 20$)

Operator	Description	Example
+	Adds two operands.	$A + B = 30$
-	Subtracts second operand from the first.	$A - B = -10$
*	Multiplies both operands.	$A * B = 200$
/	Divides numerator by de-numerator.	$B / A = 2$
%	Modulus Operator and remainder of after an integer division.	$B \% A = 0$
++	Increment operator increases the integer value by one.	$A++ = 11$
--	Decrement operator decreases the integer value by one.	$A-- = 9$

Operadores relacionales

(asumiendo que A = 10 y B = 20)

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.

Operadores lógicos

(asumiendo que $A = 1$ y $B = 0$)

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true.

En C cualquier valor $\neq 0$ es verdadero

Operadores de asignación

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	$C = A + B$ will assign the value of $A + B$ to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	$C /= A$ is equivalent to $C = C / A$

Control de flujo

```
if (boolean_expression) {  
    /* statement(s) to be executed if true */  
} else {  
    /* statement(s) to be executed if false */  
}
```

A diferencia de Python, el espacio en blanco no tiene significado.

Buclé (loops)

```
int a = 10;
while( a < 20 ) {
    printf("value of a: %d\n", a);
    a++; // a += 1;
}
a = 10;
do {
    printf("value of a: %d\n", a);
    a = a + 1;
} while( a < 20 );

for(a = 10; a < 20; a = a + 1 ){
    printf("value of a: %d\n", a);
}
```


Bitwise operations

Corrimiento izquierda o derecha

`[variable]<<[number of places]`

`[variable]>>[number of places]`

e.g.

`a >> 2`

`1 << 5`

Bitwise AND (&)

`01001000 &`

`10111000 =`

`00001000`

Bitwise XOR (^)

`01110010 ^`

`10101010`

`11011000`

Bitwise OR (|)

`01001000 |`

`10111000 =`

`11111000`

Bitwise Complement (~)

`unsigned int max =`

`~0;`

Bitwise operations

¿Dónde usar esto? Pensemos que necesitamos saber qué hay en **XX?XXXXX**, sin importar los X

XX**1**XXXXX &
00**1**00000 =

00**1**00000

XX**0**XXXXX &
00**1**00000 =

00**0**00000

Entonces, tenemos un numero diferente a cero (non-zero) solo si el bit que nos interesa es 1

```
//Ejemplo de una función
char word = ...; //1 byte
int check_if_one (int posicion)
{
    return (word & (1<<posicion));
}
```

Funciones en C

- Misma idea que en otros lenguajes de programación: “grupo de instrucciones que realizan una tarea, con una interfaz claramente descrita”.
- Cada función consta de una **declaración** y de una **definición**.
- La declaración indica el nombre de la función, el tipo de dato que retorna, y el tipo de dato de sus parámetros.

Declaración

Forma general:

```
return_type function_name (parameter list );
```

Ejemplo de una función que retorna el máximo entre dos enteros:

```
int max (int, int);
```

Opcionalmente:

```
int max (int a, int b);
```

Definición

Forma general:

```
return_type function_name( parameter list ) {  
    body of the function  
}
```

Ejemplo:

```
int max (int a, int b) {  
    int result;  
    if (a > b)  
        result = a;  
    else  
        result = b;  
    return result;  
}
```

Llamado de la función (ejemplo)

```
#include <stdio.h>

/* declaración de la función*/
int max (int a, int b);

int main (void) {
    /* definición de variables locales*/
    int a = 100;
    int b = 200;
    int ret;
    /* llamado de la funcion*/
    ret = max (a, b); // o max (b, a);
    printf ( "Max value is : %d\n", ret );
    return 0;
}
```

```
/* en el mismo archivo */

/* definición de la función */
int max (int a, int b) {
    int result;
    if (a > b)
        result = a;
    else
        result = b;
    return result;
}
```

Variables globales y locales

El **scope** de una variable es una región del programa donde una variable definida existe. Fuera de esa región no existe.

Las variables se pueden declarar:

- Dentro de una función o un bloque (variables locales)
- Fuera de todas las funciones (variables globales)

Variables locales

Solo pueden usarse por sentencias que están dentro de esa función o bloque de código. No son vistas/conocidas fuera del bloque o en otras funciones.

Variables globales

Las variables globales se definen fuera de una función, generalmente en la parte superior del programa. Mantienen sus valores a lo largo de la vida útil de su programa y se puede acceder a ellas dentro de cualquiera de las funciones definidas.

Convención: Su primera letra se escribe con mayúscula.

Arreglos

Los *arrays* (arreglos) son una estructura de datos para almacenar una colección secuencial y de tamaño fijo de elementos del mismo tipo.

Todos los arreglos consisten en ubicaciones de memoria contigua.

Declaración:

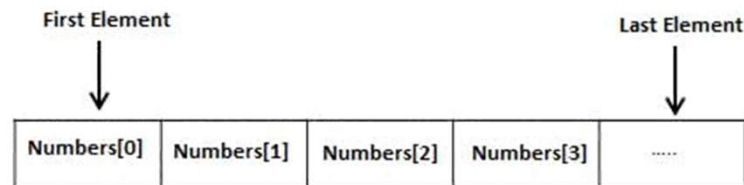
```
type array_name [array_size];
```

`array_size` debe ser una constante entera mayor que cero

`type` puede ser cualquier tipo de datos válido.

Arreglos

El primer elemento tiene índice 0



Ver <https://www.cs.utexas.edu/~EWD/transcriptions/EWD08xx/EWD831.html>

Arreglos

Ejemplo:

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

```
int n[ 10 ];  
/* initialize elements of array n */  
for (int i = 0; i < 10; i++ ) {  
    n[i] = i + 100;  
}
```

Arreglos como parámetro de una función

Tres alternativas:

```
void mi_funcion (int *param) {  
    ...  
}
```

```
void mi_funcion (int param[10]) {  
    ...  
}
```

```
void mi_funcion (int param[]) {  
    ...  
}
```

Ejemplo:

```
#include <stdio.h>  
double get_average (int arr[], int size);  
  
int main () {  
    int balance [5] = {1000, 2, 3, 17, 50};  
    double avg;  
    avg = get_average (balance, 5) ; // puntero a balance  
    return 0;  
}  
  
double get_average (int arr[], int size) {  
    double sum = 0;  
    for (int i = 0; i < size; ++i) {  
        sum += arr[i];  
    }  
    return (sum / size);  
}
```

Arreglos

¿En qué se parece y en qué se diferencian los arreglos en C con las listas en Python?

Punteros

Motivación:

¿Cómo podemos definir una función que retorne un arreglo?

Punteros

Un puntero es una variable cuyo valor es la dirección de otra variable, es decir, la dirección directa de la ubicación en la memoria.

Declaración:

```
type *var_name;
```

Ejemplo:

```
float *fp;
```

Punteros

Ejemplo:

```
#include <stdio.h>

int main (void) {
    int  var = 20;  /* variable declaration */
    int  *ip;       /* pointer declaration */

    /* store address of var in pointer*/
    ip = &var;

    printf("Address of var : %x\n", &var );
    printf("Address stored in ip : %x\n", ip );
    printf ("Value of *ip variable: %d\n", *ip );

    return 0;
}
```

El operador & entrega la dirección de una variable.

Hay dos maneras de usar un puntero:

- Sin * : Corresponde a la dirección de la variable a la que apunta.
- Con * : Corresponde al valor de la variable a la que apunta.

Aritmética de punteros

- Podemos usar ++, --, +, y -
- La operación aritmética opera en función del tamaño del tipo de datos al que apunta el puntero.

- Por ejemplo, si

```
int *ptr; // supongamos que int son 4 bytes
```

entonces

```
ptr++;
```

aumenta el valor de ptr en 4.

- ¿Cuál es la diferencia entre ptr++ y *ptr++?

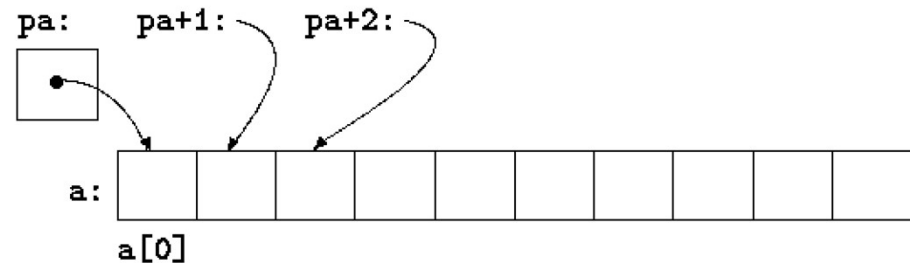
Punteros y arreglos

- El nombre de un arreglo es un puntero **constante** al primer elemento del arreglo.
- Por ejemplo, si definimos `double balance [50]`; `balance` es un puntero a `&balance[0]`, la dirección del primer elemento del arreglo.

- Ejemplo

```
double* pa;  
double a [10];  
pa = a;
```

- Podemos usar nombres de arreglos como punteros constantes, y viceversa. Por lo tanto, `*(a + 4)` es una forma de acceder a `a[4]`.
- Es decir `*pa`, `*(pa + 1)`, `*(pa + 2)`, ... son equivalentes a `a[0]`, `a[1]`, `a[2]`, ...



Punteros y arreglos

```
#include <stdio.h>
```

```
int main ( ) {  
    double arr[5] = {10.0, 2.0, 3.4, 17.0, 50.0};  
    double *p;  
    p = arr;  
    for (int i = 0; i < 5; i++ ) {  
        printf("(p + %d) : %f\n", i, *(p + i) );  
    }  
    printf("Array values using arr\n");  
    for (int i = 0; i < 5; i++ ) {  
        printf("(arr + %d) : %f\n", i, *(arr + i) );  
    }  
  
    return 0;  
}
```

Funciones: Parámetros por valor y por referencia

Por valor

```
/* function definition to swap the values */  
void swap (int x, int y) {  
  
    int temp;  
  
    temp = x; /* save the value of x */  
    x = y;    /* put y into x */  
    y = temp; /* put temp into y */  
  
    return;  
}
```

Por referencia

```
/* function definition to swap the values */  
void swap (int *x, int *y) {  
  
    int temp;  
  
    temp = *x;    /* save the value at address x */  
    *x = *y;      /* put y into x */  
    *y = temp;    /* put temp into y */  
  
    return;  
}
```

Sólo una de estas dos versiones funciona ¿Cuál?

Funciones: Parámetros por valor y por referencia

```
#include <stdio.h>

void swap (int *x, int *y);

int main ( ) {
    int a=5, b=10;
    printf("a = %d ; b = %d \n", a, b);
    swap(&a, &b); // Notar que usamos &
    printf("a = %d ; b = %d \n", a, b);
    return 0;}
```

Estructuras

- Son una manera de organizar datos relacionados entre sí.

```
struct structure_tag {  
    member definition;  
    member definition;  
    ...  
    member definition;  
} one or more structure variables;
```

- Ejemplo:

```
struct Book {  
    char  title [50];  
    char  author [50];  
    char  subject [100];  
    int   id;  
} book_1;  
book_1.id = 42;
```

Estructuras

```
#include <stdio.h>
#include <string.h>

struct Book {
    char *title;
    char author [50];
};

void printBook (struct Book);

int main( ) {
    struct Book B1; /* structure of type Book */
    B1.title = "C Programming";
    strcpy( B1.author, "Nuha Ali");

    printBook (&B1);
    return 0;
}

void printBook ( struct Book *b ) {
    printf ( "Book title : %s\n", b->title);
    printf ( "Book author : %s\n", b->author);
}
```

Igual que para cada otro tipo de datos, se pueden definir **punteros a estructuras**:

```
struct Book *struct_pointer;
```

La dirección de una estructura la obtenemos con el &:

```
struct_pointer = &B1;
```

Ahora podemos usar el puntero para acceder a los datos miembros de la estructura

```
(*struct_pointer).title = "ELO 312";
```

Con punteros a estructuras, es más conveniente usar el
-> en lugar de (*)

```
struct_pointer->title = "STM32";
```